

TP n°1 – Résolution de systèmes linéaires

Dans tout le TP, on utilisera les bibliothèques suivantes. Les questions marquées **PAPIER** se traitent sur feuille, celles marquées **PYTHON** sur machine.

TP1.py ×

```
1 import numpy as np
2 from math import *
3 import matplotlib.pyplot as plt
```

Exercice 1 – Décomposition de Cholesky et résolution de systèmes triangulaires

On s'intéresse à l'équation $A_1 X = B_1$ où

$$A_1 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 5 & 5 & 5 \\ 1 & 5 & 14 & 14 \\ 1 & 5 & 14 & 15 \end{pmatrix} \quad \text{et} \quad B_1 = \begin{pmatrix} 4 \\ 16 \\ 34 \\ 35 \end{pmatrix}.$$

PAPIER Question 1 – À l'aide de l'algorithme du pivot de Gauss, résoudre l'équation $A_1 X = B_1$.

PYTHON Question 2 – Coder la matrice A_1 et le vecteur B_1 en Python.

PYTHON Question 3 – Déterminer rapidement le spectre de A_1 .

PAPIER Question 4 – La matrice A_1 possède-t-elle une décomposition de Cholesky? Justifier votre réponse.

PYTHON Question 5 – Déterminer cette décomposition.

PYTHON Question 6 – Compléter la fonction `lower(L,B)` qui prend en argument une matrice triangulaire inférieure L et un vecteur B et renvoie le vecteur Y , solution de l'équation $LY = B$.

TP1.py ×

```
1 def lower(L, B):
2     n = len(B)
3     Y = ???
4     for i in range(n):
5         s = 0
6         for j in range(???):
7             s += ???
8         Y[i] = (B[i] - s) / ???
9     return Y
```

PYTHON Question 7 – Écrire la fonction `upper(U,Y)` qui prend en argument une matrice triangulaire supérieure U et un vecteur Y et renvoie le vecteur X , solution de l'équation $UX = Y$.

Indication `range(n-1, -1, -1)` parcourt les indices en sens inverse, de $n - 1$ jusqu'à 0.

PYTHON Question 8 – En utilisant les fonctions `upper`, `lower` et `np.linalg.cholesky`, écrire une fonction `resolvcholesky(A,B)` qui prend en argument une matrice A qui admet une décomposition de Cholesky et un vecteur B et renvoie le vecteur X , solution de l'équation $AX = B$.

PYTHON Question 9 – Retrouver la réponse de la question 1 à l'aide de la fonction `resolvcholesky`.

Exercice 2 – Méthode de Jacobi

On s'intéresse à l'équation $A_2X = B_2$ où

$$A_2 = \begin{pmatrix} 10 & 2 & -1 & 1 \\ 1 & -8 & 2 & -1 \\ 2 & 1 & 9 & -2 \\ -1 & 1 & 2 & 7 \end{pmatrix} \quad \text{et} \quad B_2 = \begin{pmatrix} 12 \\ -6 \\ 10 \\ 9 \end{pmatrix}.$$

PAPIER Question 10 – À l'aide de l'algorithme du pivot de Gauss, résoudre l'équation $A_2X = B_2$.

PYTHON Question 11 – Coder la matrice A_2 et le vecteur B_2 en Python.

PAPIER Question 12 – La méthode de Jacobi est-elle convergente pour ce système linéaire ? Justifier votre réponse.

PYTHON Question 13 – À l'aide de la fonction `np.linalg.eig`, écrire une fonction `rayonspectral(A)` qui prend une matrice A en argument et renvoie son rayon spectral.

PYTHON Question 14 – On écrit $A_2 = D + L + U$ où D est la diagonale de A_2 , L sa partie triangulaire inférieure stricte et U sa partie triangulaire supérieure stricte. Écrire trois fonctions `partdiag(A)`, `partinf(A)` et `partsup(A)` qui renvoient ces parties.

PYTHON Question 15 – Vérifier que ces trois fonctions fonctionnent correctement avec A_2 .

PYTHON Question 16 – Écrire une fonction `diagsolve(D,Y)` qui prend en argument une matrice diagonale D et un vecteur Y et renvoie le vecteur X , solution de l'équation $DX = Y$.

PYTHON Question 17 – Écrire une fonction `jacobi(A, B, tol)` qui prend en argument une matrice A , un vecteur B et une tolérance `tol`, et renvoie une approximation du vecteur X solution de $AX = B$ par la méthode de Jacobi. Le critère d'arrêt sera `np.linalg.norm(np.dot(A, X) - B) < tol`.

PYTHON Question 18 – Tester la fonction `jacobi` sur la matrice A_2 et le vecteur B_2 avec une tolérance de 10^{-6} . Vérifier que le résultat est cohérent avec la question 10.

PYTHON Question 19 – Modifier la fonction `jacobi` en une fonction `jacobi2(A, B, tol)` qui, en plus de renvoyer le vecteur solution X , renvoie le nombre d'itérations effectuées.

PYTHON Question 20 – Combien d'itérations sont nécessaires pour approcher la solution à 10^{-10} ?

Exercice 3 – Méthode de Gauss-Seidel

PYTHON Question 21 – Écrire une fonction `GaussSeidel(A, B, tol)` qui prend en argument une matrice A , un vecteur B et une tolérance `tol`, et renvoie une approximation du vecteur X solution de $AX = B$ par la méthode de Gauss-Seidel. Le critère d'arrêt sera `np.linalg.norm(np.dot(A, X) - B) < tol`.

PYTHON Question 22 – Tester la fonction `GaussSeidel` sur la matrice A_2 et le vecteur B_2 de l'exercice 2 avec une tolérance de 10^{-6} .